

"Better
Arrays
Smarter
Solutions."

NumPy



Data
Compute
Create
More

Complete Notes

Cheat Sheet + Interview Questions

From Basics to Advanced
Everything you need in one place!

Arrays
are just
the beginning...



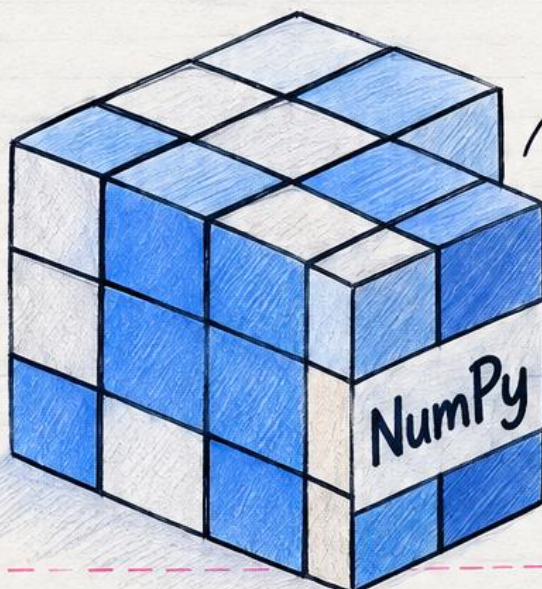
```
import  
numpy as np  
# Let's build  
something amazing!
```



Fast

Efficient

Powerful



Arrays

Math

Data

Real World
Applications



Most-Used
Functions
with examples



Practical
Examples
real-world use cases



Common
Mistakes
and how to avoid them



Performance
Tips
write faster, better code



50+ Interview
Questions
with answers



Quick
Reference
for revision

"Small Functions. Big Possibilities."



Python
+ NumPy
= Real Power

@codedsoul_05

Learn
Practice
Apply
Grow



NumPy Basics & Array Creation

"Small
Arrays,
Big Possibilities"

1

Create arrays in different ways using NumPy.

1 np.array()

Creates a NumPy array from a list or tuple.

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(a)
# [1 2 3 4]
```

2 np.zeros()

Creates an array filled with zeros.

```
import numpy as np
a = np.zeros(5)
print(a)
# [0. 0. 0. 0. 0.]
```

3 np.ones()

Creates an array filled with ones.

```
import numpy as np
a = np.ones(4)
print(a)
# [1. 1. 1. 1.]
```

4 np.empty()

Creates an uninitialized array (contains random values).

```
import numpy as np
a = np.empty(3)
print(a)
# [6.23042070e-307 1.00000000e+000 4.94065646e-324]
```

5 np.full()

Creates an array filled with a specified value.

```
import numpy as np
a = np.full((2, 3), 7)
print(a)
# [[7 7 7]
#   [7 7 7]]
```

6 np.arange()

Creates an array with evenly spaced values within a given range.

```
import numpy as np
a = np.arange(1, 10, 2) # start, stop, step
print(a)
# [1 3 5 7 9]
```

7 np.linspace()

Creates an array of evenly spaced numbers between two values.

```
import numpy as np
a = np.linspace(0, 1, 5) # start, stop, number of elements
print(a)
# [0. 0.25 0.5 0.75 1. ]
```



Array Properties & Inspection in NumPy

"Know your data, work smarter."

2

Get useful information about your NumPy arrays.

1 np.shape()

Returns the shape (dimensions) of the array.

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(np.shape(a))
# (2, 3)
```

2 np.ndim()

Returns the number of dimensions of the array.

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(np.ndim(a))
# 2
```

3 np.size()

Returns the total number of elements in the array.

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(np.size(a))
# 6
```

4 np.dtype()

Returns the data type of the elements in the array.

```
import numpy as np
a = np.array([1, 2, 3])
print(np.dtype(a))
# int64
```

5 np.itemsize()

Returns the size (in bytes) of each element.

```
import numpy as np
a = np.array([1, 2, 3])
print(np.itemsize(a))
# 8 (for int64, may vary)
```

6 np.astype()

Converts the array to a specified data type.

```
import numpy as np
a = np.array([1.5, 2.7, 3.1])
b = a.astype(int)
print(b)
# [1 2 3]
print(b.dtype)
# int64
```





Indexing & Slicing in NumPy

Access and manipulate array elements easily.

"Slice Data
Build Insights"

3

1 Basic Indexing (arr[])

Access a single element using its index (similar to Python lists).

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr[0]) # first element
print(arr[3]) # element at index 3

# Output:
# 10
# 40
```

2 Slicing (arr[start:stop])

Access a range of elements using start and stop index (stop is exclusive).

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4]) # elements from index 1 to 3
print(arr[:3]) # from start to index 2
print(arr[2:]) # from index 2 to end

# Output:
# [20 30 40]
# [10 20 30]
# [30 40 50]
```

3 Negative Indexing

Access elements from the end using negative indices.

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr[-1]) # last element
print(arr[-2]) # second last element
print(arr[-4:-1]) # from -4 to -2

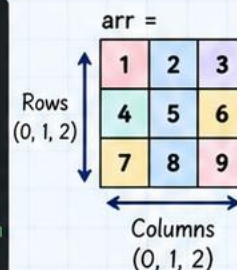
# Output:
# 50
# 40
# [20 30 40]
```

4 2D Indexing

Access elements in a 2D array using row and column indices.

```
import numpy as np
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
print(arr[0, 1]) # row 0, column 1
print(arr[2, 0]) # row 2, column 0
print(arr[1]) # entire second row
print(arr[:, 2]) # entire third column

# Output:
# 2
# 7
# [4 5 6]
# [3 6 9]
```



5 Boolean Indexing

Access elements that satisfy a condition.

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr > 25) # boolean mask
print(arr[arr > 25]) # elements greater than 25
print(arr[arr % 2 == 0]) # even elements

# Output:
# [False False True True True]
# [30 40 50]
# [10 20 30 40 50]
```

"Use conditions
to filter your data"





Reshaping & Array Manipulation in NumPy

"Same Data
Different Shape
More Possibilities"

4

Change the shape, structure, or arrangement of your arrays easily.

1 reshape()

Returns a new array with the specified shape (same data, different shape).

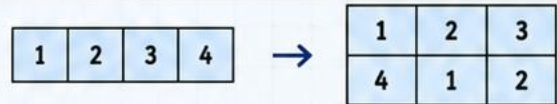
```
import numpy as np
a = np.array([1, 2, 3, 4, 5, 6])
b = a.reshape(2, 3)
print(b)
# [[1 2 3]
#  [4 5 6]]
```



2 resize()

Returns a new array with the specified shape. If needed, it repeats elements to fill the new shape.

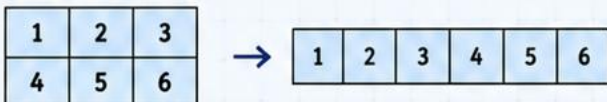
```
import numpy as np
a = np.array([1, 2, 3, 4])
b = np.resize(a, (2, 3))
print(b)
# [[1 2 3]
#  [4 1 2]]
```



3 flatten()

Returns a 1D copy of the array.

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
b = a.flatten()
print(b)
# [1 2 3 4 5 6]
```



4 ravel()

Returns a 1D view of the array (more memory efficient).

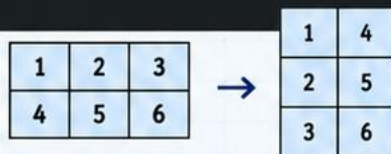
```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
b = a.ravel()
print(b)
# [1 2 3 4 5 6]
```



5 transpose()

Returns the transposed array (rows become columns).

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
b = a.transpose()
print(b)
# [[1 4]
#  [2 5]
#  [3 6]]
```



6 swapaxes()

Swaps two axes of the array.

```
import numpy as np
a = np.arange(24).reshape(2, 3, 4)
b = a.swapaxes(0, 1)
print(b.shape)
# (3, 2, 4)
```

"Rearrange
Your Array
Your Way"



Joining & Splitting Arrays in NumPy

"Combine
Data Smartly,
Split When Needed"

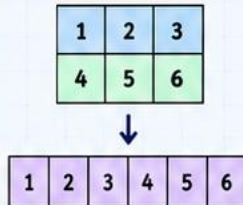
5

Combine or split arrays along different axes.

1 np.concatenate()

Joins arrays along an existing axis.

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.concatenate((a, b))
print(c)
# [1 2 3 4 5 6]
```



2 np.stack()

Joins arrays along a new axis.

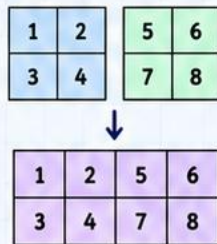
```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.stack((a, b), axis=0)
print(c)
# [[1 2 3]
#   [4 5 6]]
```

1	2	3
4	5	6

3 np.hstack()

Stacks arrays horizontally (column-wise).

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
c = np.hstack((a, b))
print(c)
# [[1 2 5 6]
#   [3 4 7 8]]
```



4 np.vstack()

Stacks arrays vertically (row-wise).

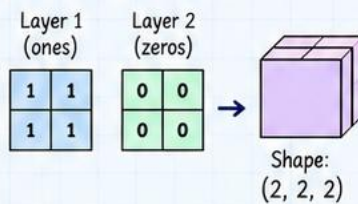
```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.vstack((a, b))
print(c)
# [[1 2 3]
#   [4 5 6]]
```

1	2	3
4	5	6

5 np.dstack()

Stacks arrays along the third axis (depth-wise).

```
import numpy as np
a = np.ones((2, 2))
b = np.zeros((2, 2))
c = np.dstack((a, b))
print(c.shape)
# (2, 2, 2)
```



6 np.split()

Splits an array into equal parts.

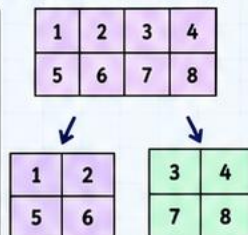
```
import numpy as np
a = np.array([1, 2, 3, 4, 5, 6])
b = np.split(a, 3)
print(b)
# [array([1, 2]),
#   array([3, 4]),
#   array([5, 6])]
```

1	2
3	4
5	6

7 np.hsplit()

Splits an array horizontally (column-wise).

```
import numpy as np
a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8]])
b = np.hsplit(a, 2)
print(b[0])
print(b[1])
# [[1 2]
#   [5 6]]
# [[3 4]
#   [7 8]]
```



8 np.vsplit()

Splits an array vertically (row-wise).

```
import numpy as np
a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9],
              [10, 11, 12]])
b = np.vsplit(a, 2)
print(b[0])
print(b[1])
# [[1 2 3]
#   [4 5 6]]
# [[7 8 9]
#   [10 11 12]]
```

1	2	3
4	5	6
7	8	9
10	11	12



Mathematical Functions in NumPy

"Simple
Functions
Powerful Results"

6

Perform element-wise mathematical operations on arrays.

1 np.add()

Adds two arrays element-wise.

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.add(a, b)
print(result)
# [5 7 9]
```

Output:
[5 7 9]

2 np.subtract()

Subtracts two arrays element-wise (a - b).

```
import numpy as np
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
result = np.subtract(a, b)
print(result)
# [9 18 27]
```

Output:
[9 18 27]

3 np.multiply()

Multiplies two arrays element-wise.

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.multiply(a, b)
print(result)
# [4 10 18]
```

Output:
[4 10 18]

4 np.divide()

Divides two arrays element-wise (a / b).

```
import numpy as np
a = np.array([10, 20, 30])
b = np.array([2, 4, 5])
result = np.divide(a, b)
print(result)
# [ 5.  5.  6.]
```

Output:
[5. 5. 6.]

5 np.power()

Raises array elements to a power.

```
import numpy as np
a = np.array([2, 3, 4])
result = np.power(a, 2)
print(result)
# [ 4  9 16]
```

Output:
[4 9 16]

6 np.sqrt()

Computes the square root of array elements.

```
import numpy as np
a = np.array([4, 9, 16])
result = np.sqrt(a)
print(result)
# [2.  3.  4.]
```

Output:
[2. 3. 4.]

7 np.abs()

Returns the absolute value of array elements.

```
import numpy as np
a = np.array([-10, -5, 3])
result = np.abs(a)
print(result)
# [10  5  3]
```

Output:
[10 5 3]

8 np.round()

Rounds array elements to the specified decimals.

```
import numpy as np
a = np.array([1.234, 2.567, 3.891])
result = np.round(a, 2)
print(result)
# [1.23 2.57 3.89]
```

Output:
[1.23 2.57 3.89]



Tip: All these functions work element-wise and support broadcasting, so you can also use a single number instead of an array.

@coded soul_05



Statistical Functions in NumPy

"Numbers
Tell a Story
Learn to Read It"

7

Get useful statistical insights from your NumPy arrays.

1 np.mean()

Returns the mean (average) of array elements.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.mean(a)
print(result)
# 30.0
```

Output:
30.0

2 np.median()

Returns the median (middle value) of array elements.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.median(a)
print(result)
# 30.0
```

Output:
30.0

3 np.std()

Returns the standard deviation of array elements.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.std(a)
print(result)
# 14.142135623730951
```

Output:
14.1421

4 np.var()

Returns the variance of array elements.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.var(a)
print(result)
# 200.0
```

Output:
200.0

5 np.min()

Returns the minimum value in the array.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.min(a)
print(result)
# 10
```

Output:
10

6 np.max()

Returns the maximum value in the array.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.max(a)
print(result)
# 50
```

Output:
50

7 np.sum()

Returns the sum of all array elements.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.sum(a)
print(result)
# 150
```

Output:
150

8 np.average()

Returns the weighted or unweighted average.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.average(a)
print(result)
# 30.0
```

Output:
30.0



Tip: `mean()` and `average()` give the same result for unweighted data.
Use `std()` and `var()` to understand data spread.



Random Functions in NumPy

"Randomness
Creates
Possibilities"

9

Generate random numbers and samples for simulations, testing and machine learning.

1 np.random()

Generates random numbers from a uniform distribution in $[0, 1)$.

```
import numpy as np
a = np.random.random(5)
print(a)
# [0.3745 0.9507 0.7319 0.5987 0.1560]
```

Output:

```
[0.3745 0.9507 0.7319 0.5987 0.1560]
(Values will be different each time)
```

2 np.rand()

Generates random numbers from a uniform distribution in $[0, 1)$ with a given shape.

```
import numpy as np
a = np.random.rand(2, 3)
print(a)
# [[0.4170 0.7203 0.0001]
#    [0.3023 0.1468 0.0923]]
```

Output:

```
[[0.4170 0.7203 0.0001]
 [0.3023 0.1468 0.0923]]
```

3 np.randn()

Generates random numbers from a normal distribution (mean = 0, std = 1).

```
import numpy as np
a = np.random.randn(2, 3)
print(a)
# [[ 0.4967 -0.1383  0.6477]
#    [ 1.5230 -0.2342 -0.2341]]
```

Output:

```
[[ 0.4967 -0.1383  0.6477]
 [ 1.5230 -0.2342 -0.2341]]
```

4 np.randint()

Generates random integers within a given range (low, high).

```
import numpy as np
a = np.random.randint(1, 10, 5)
print(a)
# [6 3 7 4 9]
```

Output:

```
[6 3 7 4 9]
(Integers from 1 to 9)
```

5 np.choice()

Randomly selects elements from an array.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
b = np.random.choice(a, 3)
print(b)
# [30 10 50]
```

Output:

```
[30 10 50]
```

(Random elements
from array)

6 np.shuffle()

Shuffles (rearranges) the elements of an array in place.

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
np.random.shuffle(a)
print(a)
# [3 5 1 4 2]
```

Output:

```
[3 5 1 4 2]
```

(Original array
is modified)

7 np.seed()

Sets a seed for random number generation (for reproducibility).

```
import numpy as np
np.random.seed(42)
a = np.random.rand(3)
print(a)
# [0.3745 0.9507 0.7319]
```

Output:

```
[0.3745 0.9507 0.7319]
```

(Same output every time with seed = 42)

"Same Seed
Same Results
Better Experiments"





Sorting & Searching in NumPy

Organize and find data quickly in your NumPy arrays.

"Find What
You Need
Faster"

8

1 np.sort()

Returns a sorted copy of the array
(in ascending order).

```
import numpy as np
a = np.array([34, 7, 23, 32, 5])
sorted_a = np.sort(a)
print(sorted_a)
# [ 5  7 23 32 34]
```

Output:
[5 7 23 32 34]

2 np.argsort()

Returns the indices that would sort the array.

```
import numpy as np
a = np.array([34, 7, 23, 32, 5])
indices = np.argsort(a)
print(indices)
# [4 1 2 3 0]
```

Output:
[4 1 2 3 0]

Index 4 (value 5)
comes first, then 1 (7),
2 (23), 3 (32), 0 (34)

3 np.searchsorted()

Finds the index where a value should be inserted
to maintain order.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
index = np.searchsorted(a, 25)
print(index)
# 2
```

Output:
2
25 should be
inserted at index 2
(to keep the array sorted)

4 np.where()

Returns indices where a condition is True.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
indices = np.where(a > 25)
print(indices)
# (array([2, 3, 4]),)
```

Output:
(array([2, 3, 4]),)

Indices of elements
greater than 25

5 np.argmax()

Returns the index of the maximum value.

```
import numpy as np
a = np.array([10, 50, 30, 20])
index = np.argmax(a)
print(index)
# 1
```

Output:
1
50 is at index 1

6 np.argmin()

Returns the index of the minimum value.

```
import numpy as np
a = np.array([10, 50, 30, 20])
index = np.argmin(a)
print(index)
# 0
```

Output:
0
10 is at index 0

7 np.nonzero()

Returns the indices of non-zero elements.

```
import numpy as np
a = np.array([0, 5, 0, 3, 7, 0])
indices = np.nonzero(a)
print(indices)
# (array([1, 3, 4]),)
```

Output:
(array([1, 3, 4]),)
Indices of non-zero elements
(5, 3, 7)

Sort Smarter
Search Faster
Work Better



Tip: `argsort()` gives indices, `argmax()` / `argmin()` give the index of max/min, `searchsorted()` helps in insertion, and `where()` / `nonzero()` help you find elements.

@coded soul_05



Linear Algebra in NumPy

“Matrices Today
Smarter Systems
Tomorrow”

10

Perform common linear algebra operations using NumPy.

1 np.dot()

Computes the dot product of two arrays (vectors or matrices).

```
import numpy as np
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])
result = np.dot(A, B)
print(result)
```

Output:
[[19 22]
[43 50]]

2 np.matmul()

Matrix multiplication (recommended for 2D arrays).

```
import numpy as np
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])
result = np.matmul(A, B)
print(result)
```

Output:
[[19 22]
[43 50]]

3 np.inner()

Computes the inner product of two arrays. (For 1D arrays it is dot product, for higher dimensions it sums over the last axis of a and the second last of b).

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.inner(a, b)
print(result)
```

Output:
32
(1*4 + 2*5 + 3*6)

4 np.outer()

Computes the outer product of two 1D arrays.

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5])
result = np.outer(a, b)
print(result)
```

Output:
[[4 5]
[8 10]
[12 15]]
(Each element $a[i]*b[j]$)

5 np.inv()

Computes the inverse of a square matrix.

```
import numpy as np
A = np.array([[1, 2],
              [3, 4]])
result = np.linalg.inv(A)
print(result)
```

Output:
[[-2. 1.]
[1.5 -0.5]]
(Inverse of A)

6 np.det()

Computes the determinant of a square matrix.

```
import numpy as np
A = np.array([[1, 2],
              [3, 4]])
result = np.linalg.det(A)
print(result)
```

Output:
-2.0
(Determinant of A)

7 np.eig()

Computes the eigenvalues and eigenvectors of a square matrix.

```
import numpy as np
A = np.array([[2, 0],
              [0, 3]])
eig_vals, eig_vecs = np.linalg.eig(A)
print(eig_vals)
print(eig_vecs)
```

Output:
Eigenvalues:
[2. 3.]
Eigenvectors:
[[1. 0.]
[0. 1.]]

8 np.solve()

Solves a system of linear equations $Ax = b$.

```
import numpy as np
A = np.array([[3, 1],
              [1, 2]])
b = np.array([9, 8])
x = np.linalg.solve(A, b)
print(x)
```

Output:
[2. 3.]
(Solution to $Ax = b$)





Broadcasting & Vectorization in NumPy

"Same Code
More Power"

11

Write cleaner code, work with arrays of different shapes and get better performance.

1 Broadcasting Rules

Broadcasting allows NumPy to perform operations on arrays with different shapes.

```
import numpy as np
a = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2, 3)
b = np.array([10, 20, 30]) # shape (3,)
result = a + b
print(result)
```

Output:

```
[[11 22 33]
 [14 25 36]]
```

Broadcasting Rules:

- Start comparing shapes from the last dimension
- Dimensions are compatible if they are equal or one of them is 1
- Missing dimensions are treated as 1
- Result shape is the maximum size along each dimension

2 How Broadcasting Works

NumPy automatically expands the smaller array without actually copying data.

```
import numpy as np
a = np.array([[1, 2, 3],
              [4, 5, 6]]) # (2, 3)
b = np.array([10, 20, 30]) # (3,)
print("a shape:", a.shape)
print("b shape:", b.shape)
print("result shape:", (a + b).shape)
```

Output:

```
a shape: (2, 3)
b shape: (3,)
result shape: (2, 3)
```

Visual Representation: b is virtually expanded to match a

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{(2,3)} + \begin{bmatrix} 10 & 20 & 30 \end{bmatrix}_{(3,)} \rightarrow \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}_{(2,3)}$$

3 Vectorized Operations

Perform operations on entire arrays without loops.

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
# add 10 to each element
a_plus = a + 10
# element-wise multiplication
b = np.array([10, 20, 30, 40, 50])
result = a * b
print(a_plus)
print(result)
```

Output:

```
[11 12 13 14 15]
[10 40 90 160 250]
```

Why use vectorization?

- Cleaner and shorter code
- Much faster than loops
- Uses optimized C libraries under the hood

4 np.vectorize()

Turns a normal Python function into a vectorized function.

```
import numpy as np
# normal Python function
def square(x):
    return x * x

# vectorize the function
vec_square = np.vectorize(square)
a = np.array([1, 2, 3, 4])
print(vec_square(a))
```

Output:

```
[ 1  4  9 16]
```

Note:

- np.vectorize() is a convenience function
- It is not as fast as true NumPy ufuncs, but useful for applying custom functions

5 Universal Functions (ufuncs)

NumPy provides fast, element-wise functions (ufuncs).

```
import numpy as np
a = np.array([1, 4, 9, 16])
print(np.sqrt(a)) # square root
print(np.exp(a)) # e^x
print(np.log(a)) # natural log
print(np.sin(a)) # sine
```

Output:

```
[1.  2.  3.  4.]
[ 2.71828183e+00  5.45981500e+01  8.10308393e+03  8.8611052e+06]
[0.  1.38629436  2.19722458  2.77258872]
[ 0.84147098 -0.7568025  0.41211849 -0.28790332]
```

6 Performance Comparison

Vectorization is much faster than Python loops.

```
import numpy as np
import time
n = 10**6
a = np.random.rand(n)
# Python loop (slow)
start = time.time()
result = [x * 2 for x in a]
print("Loop time:", round(time.time()-start, 4))
# Vectorized (fast)
start = time.time()
result = a * 2
print("Vectorized time:", round(time.time()-start, 4))
```

Output (approx):

```
Loop time: 0.2314
Vectorized time: 0.0021
```

Result:

- Vectorized operations can be 100x or more faster than Python loops
- Always prefer NumPy operations over loops



Tip: Use broadcasting and vectorization to write cleaner code, reduce loop usage and get the best performance from NumPy.

@coded soul_05



Boolean & Conditional Operations in NumPy

"Conditions
Create Clarity"

12

Filter, check conditions and work with boolean arrays efficiently.

1 Boolean Indexing

Use a boolean condition to filter elements from an array.

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
mask = a > 25
result = a[mask]
print(mask)
print(result)
```

Output:
[False False True
 True True]
[30 40 50]

2 np.where()

Returns elements based on a condition (like a ternary operator).

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = np.where(a > 25, 'High', 'Low')
print(result)
```

Output:
['Low' 'Low'
 'High' 'High' 'High']

`np.where(condition, x, y)` → returns x where condition is True, otherwise y.

3 np.any()

Returns True if at least one element satisfies the condition.

```
import numpy as np
a = np.array([0, 0, 1, 0])
result = np.any(a > 0)
print(result)

# Example with condition
b = np.array([5, 10, 15])
print(np.any(b > 12))
```

Output:
True
True

Useful to quickly check
if any element meets
a condition.

4 np.all()

Returns True if all elements satisfy the condition.

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(np.all(a > 0))

b = np.array([1, 0, 3])
print(np.all(b > 0))
```

Output:
True
False

Useful to check if
every element meets
a condition.

5 np.isnan() and np.isinf()

Check for NaN (Not a Number) and infinite values.

```
import numpy as np
a = np.array([1, np.nan, 2, np.inf, -np.inf])
print(np.isnan(a))
print(np.isinf(a))
```

Output:
[False True False False False]
[False False False True True]

Useful for cleaning and handling real world data
(where missing or infinite values can appear).

6 Filtering with Multiple Conditions

Combine multiple conditions using & (AND), | (OR), and ~ (NOT).

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
result = a[(a > 15) & (a < 45)]
print(result)

# OR condition
result2 = a[(a < 15) | (a > 45)]
print(result2)

# NOT condition
result3 = a[~(a > 25)]
print(result3)
```

Output:
[20 30 40]
[10 50]
[10 20]

Use brackets () to
combine conditions.



Tip: Boolean indexing and conditional operations make your code shorter, cleaner and much more powerful for data analysis.

@coded soul_05



Copy, View & Memory in NumPy

"Same Data
Different View
Bigger Possibilities"

13

Understand how NumPy handles memory, copying and views to write efficient code.

1 np.copy()

Creates a new array with its own independent memory.

```
import numpy as np
a = np.array([1, 2, 3])
b = a.copy()
b[0] = 100
print(a)
print(b)
```

Output:

```
[1 2 3]
[100 2 3]
```

Changes in b do not affect a.

2 np.view()

Creates a new array object that shares the same memory.

```
import numpy as np
a = np.array([1, 2, 3])
b = a.view()
b[0] = 100
print(a)
print(b)
```

Output:

```
[100 2 3]
[100 2 3]
```

Both a and b share the same data.

3 np.shares_memory()

Checks if two arrays share the same memory.

```
import numpy as np
a = np.array([1, 2, 3])
b = a.view()
c = a.copy()
print(np.shares_memory(a, b))
print(np.shares_memory(a, c))
```

Output:

```
True
False
```

a and b share memory, but a and c do not.

4 Shallow vs Deep Copy

Shallow copy shares data, deep copy creates independent data.

```
import numpy as np
import copy
a = np.array([[1, 2], [3, 4]])
shallow = copy.copy(a)
deep = copy.deepcopy(a)
shallow[0, 0] = 100
print(a)
print(shallow)
print(deep)
```

Output:

```
[[100 2]
 [ 3 4]]
[[100 2]
 [ 3 4]]
[[1 2]
 [ 3 4]]
```

Shallow copy reflects changes, deep copy does not.

5 Memory Optimization

Use smaller data types to save memory.

```
import numpy as np
a = np.array([1, 2, 3, 4], dtype=np.int64)
b = np.array([1, 2, 3, 4], dtype=np.int8)
print(a.nbytes) # 8 bytes per element
print(b.nbytes) # 1 byte per element
```

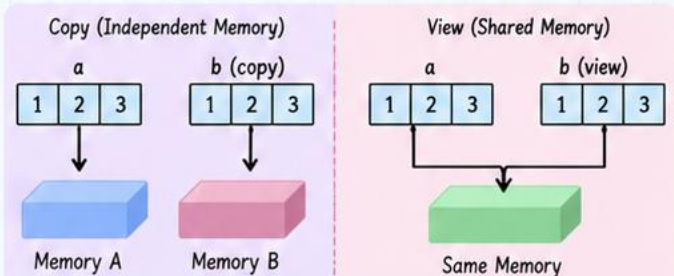
Output:

```
32
4
```

Same data, less memory with smaller dtype.

6 Visual Representation

Copy creates new memory, view shares the same memory.



Key Takeaways

- Use `copy()` when you need a completely independent array.
- Use `view()` when you want a different view of the same data (faster, uses less memory).
- Use `shares_memory()` to check if arrays share the same data.
- Choose smaller dtypes to optimize memory usage.





Advanced NumPy Functions in NumPy

"Small Functions
Big Possibilities" **14**

Useful functions for advanced data manipulation and analysis.

1 np.unique()

Returns the unique elements from an array.

```
import numpy as np
```

```
a = np.array([1, 2, 2, 3, 4, 4, 5])  
unique_vals = np.unique(a)  
print(unique_vals)
```

Output:

```
[1 2 3 4 5]
```

(Sorted unique
elements)

2 np.diff()

Computes the difference between consecutive elements.

```
import numpy as np
```

```
a = np.array([10, 15, 25, 40])  
result = np.diff(a)  
print(result)
```

Output:

```
[ 5 10 15]
```

(Differences between
elements)

3 np.gradient()

Computes the gradient (rate of change) of an array.

```
import numpy as np
```

```
a = np.array([1, 2, 4, 7, 11])  
grad = np.gradient(a)  
print(grad)
```

Output:

```
[1. 1.5 2.5 3.5 4. ]
```

(Gradient at each point)

4 np.clip()

Limits the values in an array to a specified range.

```
import numpy as np
```

```
a = np.array([-10, 0, 5, 10, 20])  
result = np.clip(a, 0, 10)  
print(result)
```

Output:

```
[ 0  0  5 10 10]
```

(Values clipped between
0 and 10)

5 np.cumsum()

Returns the cumulative sum of elements.

```
import numpy as np
```

```
a = np.array([1, 2, 3, 4, 5])  
result = np.cumsum(a)  
print(result)
```

Output:

```
[1 3 6 10 15]
```

(Cumulative sum)

6 np.cumprod()

Returns the cumulative product of elements.

```
import numpy as np
```

```
a = np.array([1, 2, 3, 4, 5])  
result = np.cumprod(a)  
print(result)
```

Output:

```
[1 2 6 24 120]
```

(Cumulative product)

7 np.repeat()

Repeats elements of an array.

```
import numpy as np
```

```
a = np.array([1, 2, 3])  
result = np.repeat(a, 3)  
print(result)
```

Output:

```
[1 1 1 2 2 2 3 3 3]
```

(Each element repeated
3 times)

8 np.tile()

Replicates an array (tiles it) a given number of times.

```
import numpy as np
```

```
a = np.array([1, 2, 3])  
result = np.tile(a, 2)  
print(result)
```

Output:

```
[1 2 3 1 2 3]
```

(Array repeated 2 times)



Tip: Combine these functions with slicing, boolean indexing
and vectorization for powerful data analysis.

@coded soul_05

"Learn the functions. Build the possibilities."



NumPy Cheat Sheet

+ Interview Questions

"Practice
Builds
Better Code"

15

Most-used functions, practical examples, common mistakes, performance tips and 50+ interview questions.

1 Array Creation

Function	Example
array()	np.array([1, 2, 3])
zeros()	np.zeros((2, 3))
ones()	np.ones((2, 3))
full()	np.full((2, 3), 5)
arange()	np.arange(0, 10, 2)
linspace()	np.linspace(0, 1, 5)
empty()	np.empty((2, 2))

2 Mathematical Functions

Function	Example
add()	np.add(a, b)
subtract()	np.subtract(a, b)
multiply()	np.multiply(a, b)
divide()	np.divide(a, b)
power()	np.power(a, 2)
sqrt()	np.sqrt(a)
abs()	np.abs(a)
round()	np.round(a, 2)

3 Statistical Functions

Function	Example
mean()	np.mean(a)
median()	np.median(a)
std()	np.std(a)
var()	np.var(a)
min()	np.min(a)
max()	np.max(a)
sum()	np.sum(a)
average()	np.average(a)

4 Sorting & Searching

Function	Example
sort()	np.sort(a)
argsort()	np.argsort(a)
searchsorted()	np.searchsorted(a, 5)
where()	np.where(a > 10)
argmax()	np.argmax(a)
argmin()	np.argmin(a)
nonzero()	np.nonzero(a)

5 Random Functions

Function	Example
random()	np.random.random((2, 2))
rand()	np.random.rand(2, 2)
randn()	np.random.randn(2, 2)
randint()	np.random.randint(1, 10, 3)
choice()	np.random.choice([1, 2, 3])
shuffle()	np.random.shuffle(a)
seed()	np.random.seed(42)

6 Linear Algebra

Function	Example
dot()	np.dot(a, b)
matmul()	np.matmul(a, b)
inner()	np.inner(a, b)
outer()	np.outer(a, b)
inv()	np.linalg.inv(a)
det()	np.linalg.det(a)
eig()	np.linalg.eig(a)
solve()	np.linalg.solve(A, b)

7 Broadcasting & Vectorization

Concept	Example
Broadcasting	a + b # different shapes
Vectorized ops	np.sin(a)
vectorize()	np.vectorize(func)
Ufuncs	np.add(), np.multiply()
Performance	Avoid loops, use vectorization

8 Boolean & Conditional Ops

Function	Example
Boolean indexing	a[a > 5]
where()	np.where(a > 0, 1, 0)
any()	np.any(a > 0)
all()	np.all(a > 0)
isnan()	np.isnan(a)
isinf()	np.isinf(a)
Filtering	a[(a > 0) & (a < 10)]

9 Copy, View & Memory

Function	Example
copy()	a.copy()
view()	a.view()
shares_memory()	np.shares_memory(a, b)
shallow copy	a.view() # shares memory
deep copy	a.copy() # independent
Memory opt.	Use smaller dtypes (e.g. int32)
Check size	a.nbytes, a.dtype

10 Advanced NumPy Functions

Function	Example
unique()	np.unique(a)
diff()	np.diff(a)
gradient()	np.gradient(a)
clip()	np.clip(a, 0, 10)
cumsum()	np.cumsum(a)
cumprod()	np.cumprod(a)
repeat()	np.repeat(a, 3)
tile()	np.tile(a, 2)

11 Practical Example

```
import numpy as np

# Create array
a = np.array([1, 2, 3, 4, 5])

# Filter even numbers and get their square
even_sq = a[a % 2 == 0] ** 2
print(even_sq)

# Calculate mean and standard deviation
print("Mean:", np.mean(a))
print("Std:", np.std(a))
```

Output:
[4 16]
Mean: 3.0
Std: 1.4142135623730951

12 Common Mistakes

- ✗ Using Python loops instead of vectorization
- ✗ Forgetting axis parameter (e.g. sum, mean)
- ✗ Modifying a view instead of a copy
- ✗ Shape mismatch in operations
- ✗ Using == for float comparison
- ✗ Not handling missing values (np.isnan)
- ✗ Using default dtype (use int32/float32)
- ✗ Ignoring broadcasting rules
- ✗ Unnecessary array copies (memory waste)

13 Performance Tips

- ✓ Use vectorized operations
- ✓ Choose appropriate dtypes
- ✓ Avoid unnecessary copies
- ✓ Use in-place operations (+, *=)
- ✓ Leverage broadcasting
- ✓ Use NumPy ufuncs
- ✓ Use memory views when possible
- ✓ Profile code with %timeit
- ✓ For large data, consider chunking
- ✓ Use NumPy built-in functions

14 50+ Interview Questions

- What is NumPy and why is it used?
- Difference between array and list?
- What are ufuncs?
- Explain broadcasting with example
- Difference between view and copy?
- How does np.where() work?
- What is the difference between np.sum() and np.add()?
- Explain axis in NumPy.
- How to handle missing values?
- What is vectorization?
- How to improve performance?
- Difference between np.dot() and np.matmul()?
- What does np.unique() do?
- Explain np.gradient()
- How to check memory usage?
- What is the difference between np.sort() and np.argsort()?
- How does broadcasting work with different shapes?
- Explain np.linalg functions.
- How to generate random numbers?
- And 30+ more questions...



NumPy Tip: Think in arrays, not loops. Vectorization makes your code cleaner, faster and more powerful.

"Better arrays, smarter solutions."

@coded soul_05